PERSONAL EXPENSE TRACKER

A PROJECT REPORT

Submitted by

SAKTHIVEL R

(2116241501177)

in partial fulfilment of the End Semester Examination for the course

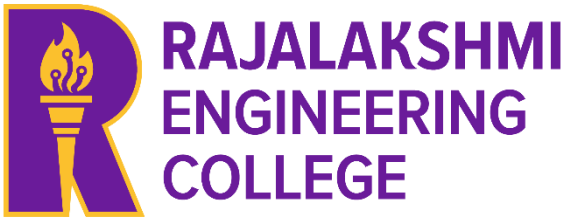
CB23F35 –

LOW-CODE APPLICATION DEVELOPMENT USING ZOHO CREATOR

in

BACHELOR OF TECHNOLOGY

Artificial Intelligence and Machine Learning



RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

Rajalakshmi Nagar, Thandalam, Tamil Nadu – 602 105

2026-2027

RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

BONAFIDE CERTIFICATE

Certified that this project report titled "PERSONAL EXPENSE TRACKER" is the bonafide work of SAKTHIVEL R (2116241501177), who carried out the individual project work on the Zoho Creator low-code platform for the course CB23F35 – Low-Code Application Development using Zoho Creator, during the academic year 2026–2027, Semester V, under my supervision.

SIGNATURE

Dr Ayyadurai M
HEAD OF THE DEPARTMENT
Department of Artificial Intelligence
And Machine Learning
Rajalakshmi Engineering College
Thandalam – 602 105

SIGNATURE

Ms Arulmozhi
COURSE FACULTY
Department of Artificial Intelligence
And Machine Learning
Rajalakshmi Engineering College
Thandalam – 602 105

Submitted for the End Semester Examination (Individual Project) held on _____.

INTERNAL EXAMINER

EXTERNAL EXAMINER

DECLARATION

I, Sakthivel R (2116241501177), declare that this project report titled “PERSONAL EXPENSE TRACKER,” submitted for the End Semester Examination of CB23F35 – Low-Code Application Development using Zoho Creator, records the individual project work completed by me. The application described here was built in Zoho Creator, with the initial structure generated using Zia AI and then reviewed and refined by me. Any external references used are cited in the report.

I further declare that this work has not been submitted, in part or in full, for the award of any other degree or course credit.

Place: Chennai
Date: 21 October 2026

SIGNATURE OF THE STUDENT
Sakthivel R
2116241501177

ACKNOWLEDGEMENT

I express my sincere gratitude to our Chairman, Mr Meganathan M, Chairperson, Ms Thangam Meganathan and Vice-Chairman, Mr Abhay Meganathan for providing the infrastructure and facilities that made this work possible.

I am grateful to our Principal, Dr Murugesan SN, Rajalakshmi Engineering College, for the constant encouragement extended to students to take up industry-relevant learning.

I thank, Dr Ayyadurai M, Head of the Department, Department of Artificial Intelligence and Machine Learning for the guidance and support provided throughout the course.

I sincerely thank my course faculty, Ms Arulmozhi, for her guidance, reviews and feedback during the development of this application and report.

I also acknowledge Zoho Corporation / Zoho Creator Education Programme for providing access to the Zoho Creator platform and learning resources, and I thank my family and friends for their unwavering support.

Sakthivel R

ABSTRACT

Tracking personal expenses in notebooks or loose spreadsheets makes it hard to know how much has been spent in a month, which category is using the most money, and whether the planned budget is about to be crossed. Totals are calculated by hand, mistakes are common, and no one warns the user until the month has ended.

The Personal Expense Tracker was built in Zoho Creator with the help of Zia AI. A detailed natural-language prompt was used to generate the first version of the application, which was then reviewed and refined. The application has three forms – User Profile, Expense Tracker and Monthly Budget – together with reports and a dashboard that shows KPI tiles, charts and an expense status panel calculated from the stored records. Deluge scripts validate the data, prevent duplicate profiles, calculate monthly spending, budget utilisation and status, and send email notifications for registration, expense entry and budget thresholds.

The report documents the requirements, design, implementation, testing, deployment, security and governance of the application and maps it to the fifteen laboratory experiments of the course. The welcome email workflow and the populated reports and dashboard are shown with screenshots. Remaining tests, permission settings and optional integrations are marked clearly so that only work that has actually been done is reported as completed.

Keywords: Low-Code Development; Zoho Creator; Zia AI; Expense Tracker; Deluge; Workflow Automation; Budget Alerts

PROJECT AT A GLANCE

Attribute	Details
Project Title	Personal Expense Tracker
Application Domain	Personal finance management (expense and budget tracking)
Student Name & Register No.	Sakthivel R – 2116241501177
Zoho Creator Edition	Trial edition (15 days remaining when the screenshots were taken)
Application Name (Link Name)	Personal Expense Tracker (Personal_Expense_Tracker)
Published / Live URL	[Add the application URL here]
Demo Credentials (per role)	Not included; use assigned Creator accounts
Number of Forms / Subforms	3 forms / no subforms
Number of Reports (by type)	List, grouped and summary reports (see Table 5.1)
Number of Pages / Dashboards	1 dashboard page
Number of Workflows	Registration, expense and budget form workflows
Deluge Functions / Custom Actions	2 custom functions: updateBudgetStatus, getCategorySummary
Integrations	Built-in email; optional read-only REST API test
Roles & Profiles Defined	Administrator and Regular User
Environments Used	Trial account (single environment)
Date of Production Deployment	Not deployed to production

TABLE OF CONTENTS

ABSTRACT iv
PROJECT AT A GLANCE v
LIST OF TABLES viii
LIST OF FIGURES ix
LIST OF ABBREVIATIONS x
INTRODUCTION 1
1.1 Background 1
1.2 Problem Statement 1
1.3 Objectives 2
1.4 Scope of the Project 2
1.5 Justification for a Low-Code Approach 2
1.6 Organisation of the Report 3
LITERATURE SURVEY AND SYSTEM STUDY 4
2.1 Study of the Existing System 4
2.2 Limitations of the Existing System 4
2.3 Low-Code versus Traditional Development 4
2.4 Literature Survey 5
2.5 Emerging Trends in Low-Code Development 6
2.6 Proposed System 6
REQUIREMENT ANALYSIS 7
3.1 Stakeholders and User Roles 7
3.2 Functional Requirements 7
3.3 Non-Functional Requirements 8
3.5 Use Case Model 9
SYSTEM DESIGN 10
4.1 System Architecture 10
4.2 Data Model 10
4.3 Form Design 11
4.4 Process and Workflow Design 13
4.5 User Interface and Navigation Design 13
4.6 Access Control Design 14
IMPLEMENTATION 15
5.1 Application Setup 15
5.2 Forms and Data Capture 15
5.3 Reports 17
5.4 Pages and Dashboards 18
5.5 Deluge Scripting 19
5.6 Workflows and Automation 20
5.7 Integrations 22

5.8 User Management and Role-Based Access 22
5.9 AI and Analytics Features 22
5.10 Pro-Code Extension 23
TESTING AND DEBUGGING 24
6.1 Testing Strategy 24
6.2 Test Cases 24
6.3 Debugging Deluge Scripts 25
6.4 Defect Log 25
6.5 User Acceptance Testing 26
DEPLOYMENT, SECURITY AND GOVERNANCE 27
7.1 Environments and Deployment Strategy 27
7.2 Performance Optimisation 27
7.3 Security Features 28
7.4 Governance, Backup and Maintenance 28
RESULTS AND DISCUSSION 29
8.1 Application Output 29
8.2 Achievement of Objectives 30
8.3 Comparison with the Existing System 30
8.4 Low-Code Limitations Case Study 30
8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum 31
CONCLUSION AND FUTURE ENHANCEMENTS 32
9.1 Conclusion 32
9.2 Future Enhancements 32
9.3 Learning Outcomes and Career Relevance 32
REFERENCES 34
APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST 35
APPENDIX B – COURSE OUTCOME AND EXPERIMENT MAPPING 36
APPENDIX C – COMPLETE DELUGE SOURCE CODE 37
APPENDIX D – USER MANUAL 40
D.1 Accessing the Application 40
D.2 Instructions for Regular Users 40
D.3 Instructions for Administrators 40
D.4 Setting up Roles and Record-Level Access 40
APPENDIX E – EVALUATION RUBRIC 41
APPENDIX F – VIVA-VOCE RECORD SHEET 43

LIST OF TABLES

Table 2.1: Comparison of Low-Code and Traditional Development for the Expense Tracker 5
Table 2.2: Summary of Literature Survey 5
Table 3.1: Stakeholders and User Roles 7
Table 3.2: Functional Requirements 7
Table 3.3: Non-Functional Requirements 8
Table 3.4: Hardware and Software Requirements 8
Table 3.5: Use Case Description – Add an Expense 9
Table 4.1: Entities and Relationships 11
Table 4.2: Field Specification – User Profile Form 11
Table 4.3: Field Specification – Expense Tracker Form 12
Table 4.4: Field Specification – Monthly Budget Form 12
Table 4.5: Role–Permission Matrix 14
Table 5.1: Reports in the Application 17
Table 5.2: Deluge Scripts 19
Table 5.3: Workflows 20
Table 5.4: Integrations 22
Table 6.1: Functional and Workflow Test Cases 24
Table 6.2: Defect Log 25
Table 6.3: User Acceptance Feedback 26
Table 7.1: Deployment Checklist 27
Table 7.2: Performance Optimisation Practices 27
Table 7.3: Security Controls 28
Table 8.1: Mapping of Objectives to Outcomes 30
Table 8.2: Existing System versus Proposed Application 30
Table A.1: Feature Coverage Checklist 35
Table B.1: Mapping of Course Outcomes to Report Chapters 36
Table B.2: Mapping of the Application to the 15 Laboratory Experiments 36
Table E.1: Evaluation Rubric for the Individual Project 41
Table E.2: Performance Level Descriptors 42
Table F.1: Viva-Voce Questions and Responses 43
Table F.2: Marks Summary 43

LIST OF FIGURES

Figure 2.1: Manual Expense Tracking Process Used as the Baseline 4
Figure 3.1: User Roles and Main Application Use Cases 9
Figure 4.1: Zoho Creator Application Architecture 10
Figure 4.2: Core Entity Relationships in the Application 11
Figure 4.3: Registration, Expense and Budget Alert Processes 13
Figure 4.4: Application Navigation Structure 14
Figure 5.1: Application Dashboard and Navigation 15
Figure 5.2: User Profile Form 16
Figure 5.3: Expense Tracker Form 16
Figure 5.4: Monthly Budget Form 16
Figure 5.5: All Monthly Budgets Report 17
Figure 5.6: Expense Reports 18
Figure 5.7: Monthly Income Summary of the User Profile Report 18
Figure 5.8: Updated Dashboard 19
Figure 5.9: Workflow Configuration 21
Figure 5.10: Welcome Email Received After Registration 21
Figure 5.11: Expense Confirmation and Budget Alert Emails 21
Figure 5.13: Roles and Permissions 22
Figure 5.14: Zia AI Application Generation and Spending Summary 23
Figure 6.1: Validation Error Message 25
Figure 8.1: Dashboard with KPI Tiles, Chart and Expense Status 29
Figure 8.2: Monthly Budgets with Colour-Coded Status 29
Figure 8.3: Expense Records Report 29

LIST OF ABBREVIATIONS

This list defines abbreviations used in this report.

Abbreviation	Expansion
AI	Artificial Intelligence
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
ER	Entity–Relationship
INR	Indian Rupee
JSON	JavaScript Object Notation
KPI	Key Performance Indicator
LCAP	Low-Code Application Platform
RBAC	Role-Based Access Control
REST	Representational State Transfer
UAT	User Acceptance Testing
UI	User Interface
Zia	Zoho's AI assistant
Zoho Creator	Low-code application platform

CHAPTER 1
INTRODUCTION

1.1 Background

Managing personal money is a routine task for every student and working adult. Income arrives monthly, but spending is spread across food, transport, shopping, education, bills, healthcare and entertainment, and is paid through cash, UPI and cards. When expenses are noted in a notebook, a phone message or a loose spreadsheet, a person rarely knows how much has been spent in the current month, which category is consuming the most money, or whether the monthly budget is about to be crossed.

This project builds a small **Personal Expense Tracker** on the Zoho Creator low-code platform. The application stores user profiles, individual expense records and monthly budgets, sends email notifications on registration, expense entry and budget thresholds, and presents the data through reports and a dashboard. The initial structure of the application was generated by giving a detailed natural-language prompt to Zoho Creator's Zia AI app builder, and the generated forms, reports and dashboard were then reviewed and refined. The demonstration data in the application is fictional and is not real financial information.

1.2 Problem Statement

Individuals who track spending manually lack a single, consistent record of income, expenses and budgets, and receive no timely warning when spending approaches or crosses the planned limit. The project addresses this by providing a structured, shareable, low-code application with automated email alerts and a dashboard summary. The main pain points are:

- Expenses, income and budgets are stored in separate notes or sheets, so totals are calculated by hand and are often wrong.
- There is no automatic warning when the monthly budget is close to being used up or has been exceeded.
- Category-wise spending is hard to see, so high-spending areas are noticed only after the month ends.
- Records of different people cannot be kept apart safely when one shared file is used.

1.3 Objectives

- To create three related forms – User Profile, Expense Tracker and Monthly Budget – with suitable field types and validation.
- To send a registration email, an expense confirmation email and a monthly budget alert email using Zoho Creator email actions and Deluge.
- To calculate monthly spending, budget utilisation, remaining budget and budget status from the stored records.
- To provide reports and one dashboard with KPI tiles, charts and status panels calculated from actual records.
- To use Zia AI for generating the application structure and to document how its output was reviewed, together with a clearly labelled rule-based alternative where native AI analysis is not available.
- To document access control, testing, deployment and governance, and to map the application to all fifteen laboratory experiments of the course.

1.4 Scope of the Project

The project is deliberately small so that it can be demonstrated in a college laboratory. It contains exactly three primary forms, a set of reports, one dashboard, email workflows and Deluge scripts. Payment gateways, bank-account connections, complex authentication and enterprise modules are outside the scope. The application is built and tested in a Zoho Creator trial account; the screenshots in this report show the web interface. Mobile-specific behaviour, customer-portal access and production release are not part of the verified scope.

1.5 Justification for a Low-Code Approach

Zoho Creator suits this project because forms, lookups, reports and dashboards can be created visually, while Deluge scripting adds validation, calculation and email logic exactly where needed. Zia AI shortens the initial design by turning a written requirement into forms and reports. Managed hosting, built-in authentication and the sendmail action remove the need to build a separate web stack, database and mail service. The platform's capabilities and limits are discussed in the literature [1], [2] and in the vendor documentation [3]–[5].

1.6 Organisation of the Report

Chapter 2 studies the manual baseline, compares low-code with traditional development and summarises related literature and trends. Chapter 3 records the requirements. Chapter 4 presents the system design. Chapter 5 describes the implementation in Zoho Creator. Chapter 6 covers testing and debugging. Chapter 7 covers deployment, security and governance. Chapter 8 discusses results and limitations. Chapter 9 concludes the report and lists future enhancements. The appendices contain the feature checklist, course-outcome and experiment mapping, Deluge source code, a user manual and the examiner sheets.

CHAPTER 2
LITERATURE SURVEY AND SYSTEM STUDY

2.1 Study of the Existing System

The baseline for comparison is the common manual method of tracking personal expenses. No measured data was collected about this baseline; it is described as a working assumption for analysis. A person pays for something, tries to remember it later, writes it in a notebook or a spreadsheet, adds up the totals at the end of the month and compares them with a budget kept in a separate place. Alerts depend entirely on the person remembering to check.

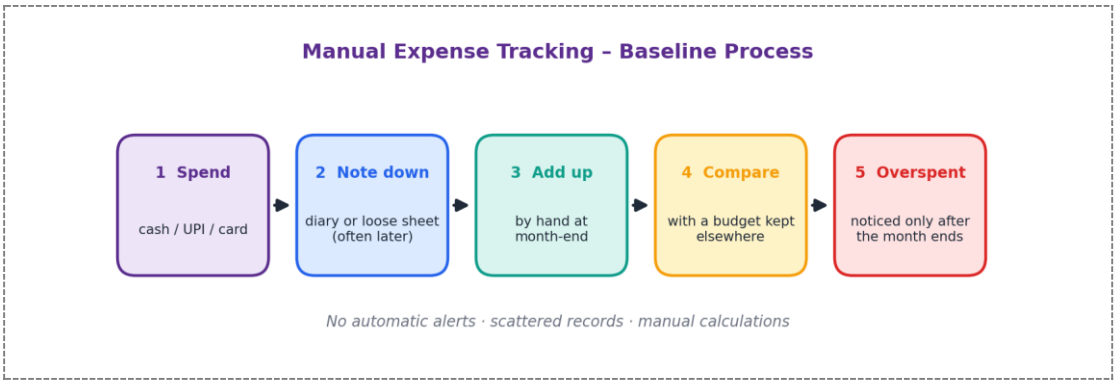


Figure 2.1: Manual Expense Tracking Process Used as the Baseline

2.2 Limitations of the Existing System

- Records are scattered and easily forgotten, duplicated or mistyped.
- Monthly totals, category totals and budget percentages must be calculated manually.
- No automatic warning is raised when spending reaches 80% or 100% of the budget.
- Privacy and separation of different users' data are not enforced.

2.3 Low-Code versus Traditional Development

A traditional implementation of this tracker would need a database schema, a back-end service, a web front end, authentication, an email service and a hosting pipeline. In Zoho Creator, forms, reports, dashboards, authentication and email actions are configured visually and Deluge is used only where custom logic is needed. The comparison below is qualitative because no development-time measurement was recorded.

Table 2.1: Comparison of Low-Code and Traditional Development for the Expense Tracker

Criterion	Traditional (Pro-Code) Development	Low-Code on Zoho Creator
Development effort	Separate design and coding of database, API, UI, authentication and email; typically weeks for the same scope.	Forms, reports and dashboard generated by Zia and refined visually; Deluge only for validation and alerts.
Skills required	Programming language, framework, database, web hosting and deployment skills.	Creator configuration, data modelling and basic Deluge scripting.
Flexibility	Very high; any architecture or UI can be built.	High within the platform; limited to Creator components, report types and Deluge capabilities.
Hosting and infrastructure	Chosen, secured and maintained by the developer.	Managed by Zoho as part of the platform.
Security and access control	Must be designed and coded (login, roles, record rules).	Built-in users, permission profiles and record-level criteria; must still be configured and tested.
Maintenance	Code changes, rebuilds and redeployments for every change.	Forms, fields and reports edited visually; scripts edited in place.
Cost	Engineering time, hosting and licences.	Platform subscription (trial used here); lower build effort.

2.4 Literature Survey

The survey uses two peer-reviewed review papers and three Zoho Creator documentation sources. The literature supports a balanced view: low-code can speed up delivery, but complexity, governance and platform dependence remain design considerations [1], [2]. Zoho's documentation describes forms, workflows and Deluge as the configurable mechanisms for data capture and automation [3]–[5].

Table 2.2: Summary of Literature Survey

S.No.	Author(s) / Source & Year	Title / Product	Key Findings	Relevance to this Project
1	Gao et al., 2026	Systematic review of low-code viability	Reports development-speed benefits with context-dependent quality and complexity outcomes.	Supports a balanced evaluation of the tracker; does not prove its performance.
2	Ajimati et al., 2025	Review of low-code / no-code adoption	Reviews adoption drivers and challenges, including governance and platform dependence.	Frames platform lock-in and governance considerations.
3	Zoho Corporation, 2026	Zoho Creator Quickstart Guide	Introduces forms, reports, workflows and application building in Creator.	Direct platform documentation for the project.

S.No.	Author(s) / Source & Year	Title / Product	Key Findings	Relevance to this Project
4	Zoho Corporation, 2026	Understanding workflows	Describes workflow types and event-driven automation.	Supports the form workflows used for emails and validation.
5	Zoho Corporation, 2026	Creating blueprints	Explains stages, transitions and actions in a blueprint.	Background for process-based extensions; not used in this small tracker.

2.5 Emerging Trends in Low-Code Development

Several trends are relevant to this project. **Citizen development** allows people without a software background, such as the student building this tracker, to create working business applications with visual builders. **AI-assisted development** lets a developer describe an application in plain language and have the platform propose forms, fields, reports and scripts; in this project Zia AI produced the first version of the application from a written prompt. **Automation** through workflows and scheduled tasks replaces manual follow-up, as seen in the automatic emails. **Governance and security** are becoming more important as non-specialists create applications that handle personal data.

AI-generated output must be reviewed; no AI feature beyond application generation is claimed without a screenshot.

2.6 Proposed System

The proposed Personal Expense Tracker centralises three record types. The **User Profile** form registers a person and triggers a welcome email. The **Expense Tracker** form records each expense with category, amount, date and payment method, validates the amount and sends a confirmation email. The **Monthly Budget** form stores the monthly and category budgets, and Deluge logic calculates the monthly spending, utilisation percentage, remaining amount and status (Within Budget, Near Limit or Exceeded). Warning emails are sent when spending first reaches 80% and when it exceeds 100% of the budget. A dashboard presents KPI tiles, charts and recent records calculated from the stored data.

CHAPTER 3
REQUIREMENT ANALYSIS

3.1 Stakeholders and User Roles

Table 3.1: Stakeholders and User Roles

Role	Description	Key Responsibilities in the Application
Administrator	Owner of the application; identified by the Is Admin option in the User Profile form.	Manage profiles, expenses, budgets, reports and application settings.
Regular User	A person who tracks personal expenses.	Maintain own profile, add own expenses, set own monthly budgets and view own reports.

3.2 Functional Requirements

The requirements below describe the observable behaviour expected from the application.

Table 3.2: Functional Requirements

FR ID	Requirement	Role(s)	Priority
FR-01	Register a user profile with name, email, monthly income, registration date and preferred currency (default INR); prevent duplicate email profiles.	Administrator, User	High
FR-02	Send a welcome email to the registered email address after a profile is saved.	System	High
FR-03	Record an expense with title, category, amount, date, payment method, description and owner.	User	High
FR-04	Validate that the expense amount is greater than zero and default the date to today.	System	High
FR-05	Send an expense confirmation email containing the expense details.	System	High
FR-06	Record a monthly budget with total and category budgets for a month.	User	High
FR-07	Calculate monthly spending, utilisation percentage, remaining budget and budget status.	System	High
FR-08	Send a budget warning at 80% and an overspending alert above 100%, without repeating the same alert.	System	High
FR-09	Provide reports for all expenses, current-month expenses, category-wise expenses, budget summary and user profiles.	Administrator, User	Medium

FR ID	Requirement	Role(s)	Priority
FR-10	Provide a dashboard with KPIs, charts and a status panel calculated from actual records.	Administrator, User	Medium
FR-11	Restrict regular users to their own records; administrators can manage all records.	Administrator, User	High

3.3 Non-Functional Requirements

Table 3.3: Non-Functional Requirements

NFR ID	Category	Requirement
NFR-01	Performance	Reports and dashboard should open quickly for the demonstration data volume; reports are filtered by month and user.
NFR-02	Security	Access must be controlled using Creator permission profiles and record-level criteria, not only by hiding fields.
NFR-03	Usability	Simple forms with clear labels, mandatory-field checks and success messages suitable for beginners.
NFR-04	Availability	Depends on the Zoho Creator service; no project-specific target is set.
NFR-05	Maintainability	Consistent form and field link names; a small number of scripts that are easy to read and change.
NFR-06	Privacy	Only fictional demonstration data is used; no private financial data is published.

The application is implemented in Zoho Creator and accessed through a modern web browser. Deluge is the scripting language used by Creator.

Table 3.4: Hardware and Software Requirements

Category	Specification
Platform	Zoho Creator (trial edition at the time of development)
AI assistance	Zia AI app builder (application generation from a prompt)
Scripting language	Deluge
Browser	Current web browser (Chrome used for the screenshots)
Email	Creator built-in email (sendmail); default notification sender of the trial account
API testing tool	Postman (optional, for the REST API demonstration)
Client hardware	Internet-connected computer capable of running a modern browser

3.5 Use Case Model

The use case model has two actors, the Administrator and the Regular User. Both can interact with the three forms, reports and the dashboard, but the Regular User sees only personal records.

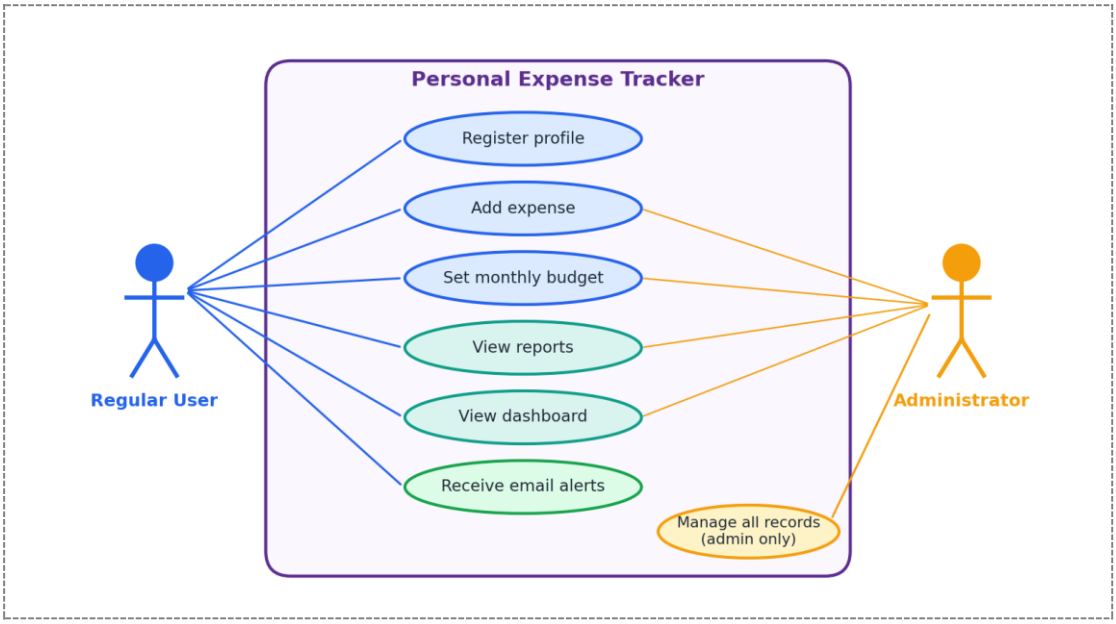


Figure 3.1: User Roles and Main Application Use Cases

Table 3.5: Use Case Description – Add an Expense

Use Case ID	UC-01
Use Case Name	Add an expense and receive confirmation
Actor(s)	Regular User
Pre-condition	User has a registered profile and is signed in to the application.
Main Flow	1. Open the Expense Tracker form. 2. Enter title, category, amount, date, payment method and description. 3. Submit. 4. The amount is validated. 5. The record is saved and a success message is shown. 6. A confirmation email is sent. 7. The monthly budget status is recalculated.
Alternate Flow	If the amount is zero or negative, an error message is shown and the record is not saved.
Post-condition	The expense appears in the expense reports and in the dashboard figures.
Related FR	FR-03, FR-04, FR-05, FR-07, FR-08

CHAPTER 4
SYSTEM DESIGN

4.1 System Architecture

Users access the application through the Zoho Creator web interface. Forms capture data, which is stored in the Creator data layer. Form workflows written in Deluge validate input, calculate budget values and send emails. Reports and the dashboard read the stored records and present them to the user. No external service is required for the core application.

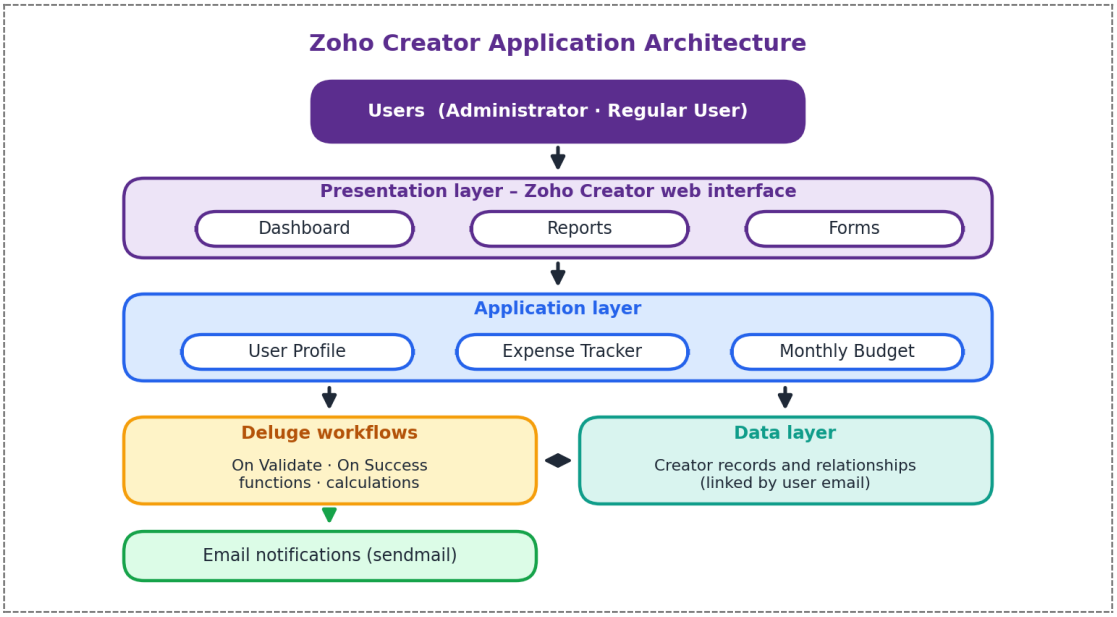


Figure 4.1: Zoho Creator Application Architecture

The system has three logical layers: (1) the presentation layer, which is the Creator web interface with the dashboard, forms and reports; (2) the application layer, which holds the forms, reports, dashboard and Deluge workflows; and (3) the data layer, where Creator stores the form records and relationships.

4.2 Data Model

The three forms are related through the user's email address. One user can have many expenses and many monthly budgets. Monthly spending is calculated by summing the Expense Tracker records of a user for the month of the budget.

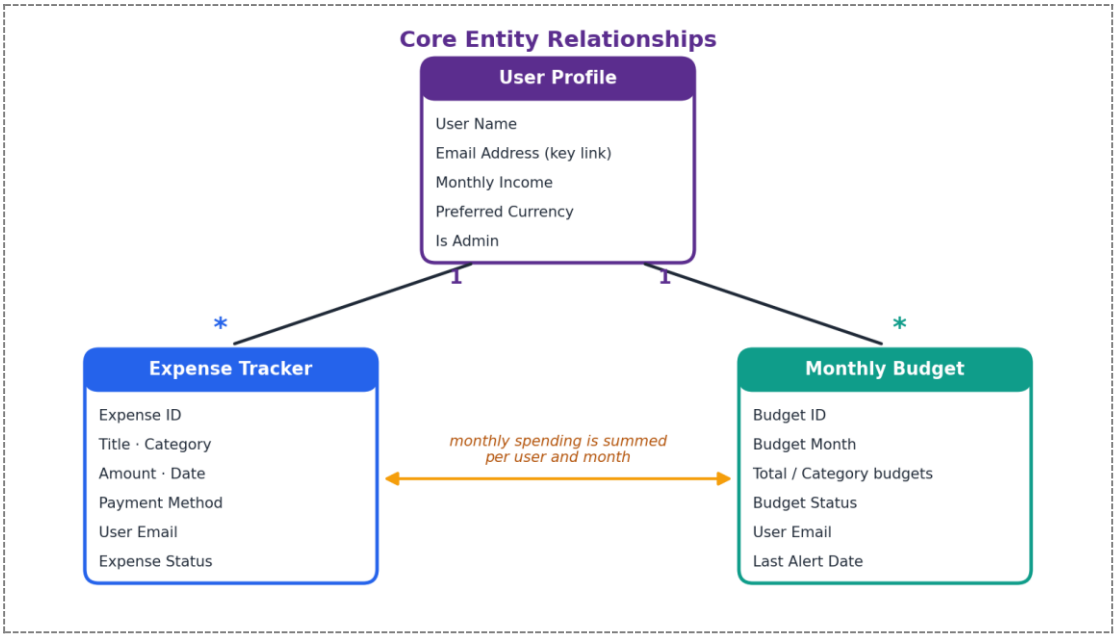


Figure 4.2: Core Entity Relationships in the Application

Table 4.1: Entities and Relationships

Entity (Form)	Key Fields	Related To	Relationship	Implemented Using
User Profile	User Name, Email Address, Monthly Income, Registration Date, Preferred Currency, Phone Number, Address, Account Status, Is Admin	Expense Tracker, Monthly Budget	One user to many expenses and budgets	User email field
Expense Tracker	Expense ID, Title, Category, Amount, Date, Payment Method, Description, User Email, Status	User Profile	Many expenses belong to one user	User email / owner field
Monthly Budget	Budget Month, Total Budget Amount, Food / Transport / Shopping / Other Budgets, Budget Status, User Email, Last Alert Date, Budget Notes	User Profile, Expense Tracker	One budget per user per month; spending read from expenses	User email field and Deluge fetch

4.3 Form Design

The application has exactly three forms. The field list of the User Profile form and the columns of the Monthly Budget report were taken from the screenshots of the built application; the Expense Tracker fields follow the project specification. Link names are the names Creator generates from the field labels and should be checked in the form builder before the scripts in Appendix C are used.

Table 4.2: Field Specification – User Profile Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default
User Name	User_Name	Name	Yes	First name shown in the welcome email

Field Label	Link Name	Field Type	Mandatory	Validation / Default
Email Address	Email_Address	Email	Yes	Duplicate email blocked by Deluge on validate
Monthly Income	Monthly_Income	Currency	No	Optional; numeric
Registration Date	Registration_Date	Date	No	Populated automatically with today's date
Preferred Currency	Preferred_Currency	Drop Down	No	Default INR
Phone Number	Phone_Number	Phone (+91)	No	Shown in the welcome email
Address	Address	Address	No	Address line 1
Account Status	Account_Status	Drop Down	No	Account state of the user
Is Admin	Is_Admin	Decision Box	No	Marks an administrator

Table 4.3: Field Specification – Expense Tracker Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default
Expense ID	Expense_ID	Auto Number	Auto	Generated automatically
Expense Title	Expense_Title	Single Line	Yes	Short description of the expense
Expense Category	Expense_Category	Drop Down	Yes	Food, Transport, Shopping, Education, Bills, Healthcare, Entertainment, Other
Amount	Amount	Currency / Decimal	Yes	Must be greater than zero
Expense Date	Expense_Date	Date	Yes	Defaults to today if empty
Payment Method	Payment_Method	Drop Down	Yes	Cash, UPI, Debit Card, Credit Card, Other
Description	Description	Multi Line	No	Optional
User Email	User_Email	Email / Lookup	Yes	Associates the expense with the owner
Expense Status	Expense_Status	Drop Down	No	Confirmed, Pending or Rejected (shown on the dashboard)

Table 4.4: Field Specification – Monthly Budget Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default
Budget ID	Budget_ID	Auto Number	Auto	Generated automatically
User Email	User_Email	Email / Lookup	Yes	Owner of the budget
Budget Month	Budget_Month	Date (month)	Yes	First day of the month
Total Budget Amount	Total_Budget_Amount	Currency	Yes	Must be greater than zero
Food / Transport / Shopping / Other Budget	Food_Budget, Transport_Budget, Shopping_Budget, Other_Budget	Currency	No	Optional category budgets

Field Label	Link Name	Field Type	Mandatory	Validation / Default
Budget Status	Budget_Status	Drop Down	No	Within Budget, Near Limit, Exceeded; set by script
Last Alert Date	Last_Alert_Date	Date	No	Date the last alert was sent; avoids repeats
Budget Notes	Budget_Notes	Multi Line	No	Free text

4.4 Process and Workflow Design

Three automated processes are designed. On registration, a form workflow checks for a duplicate email and then sends the welcome email. On expense submission, the amount is validated, the date is defaulted, the confirmation email is sent and the budget status for that user and month is recalculated. On the budget side, the status is recalculated whenever a budget or an expense is saved, and an alert email is sent only when the status changes to Near Limit or Exceeded.

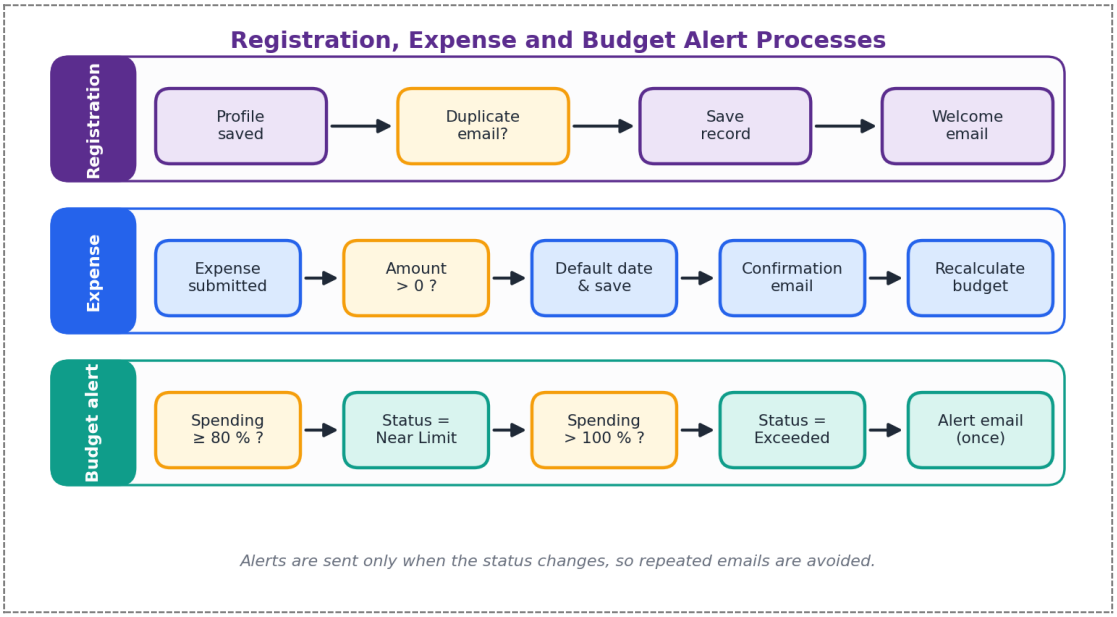


Figure 4.3: Registration, Expense and Budget Alert Processes

4.5 User Interface and Navigation Design

The application menu contains four entries: Dashboard, Expense Records, User Profiles and Monthly Budgets. The dashboard provides quick links for adding a new expense record, user profile and monthly budget and for showing all expense records.

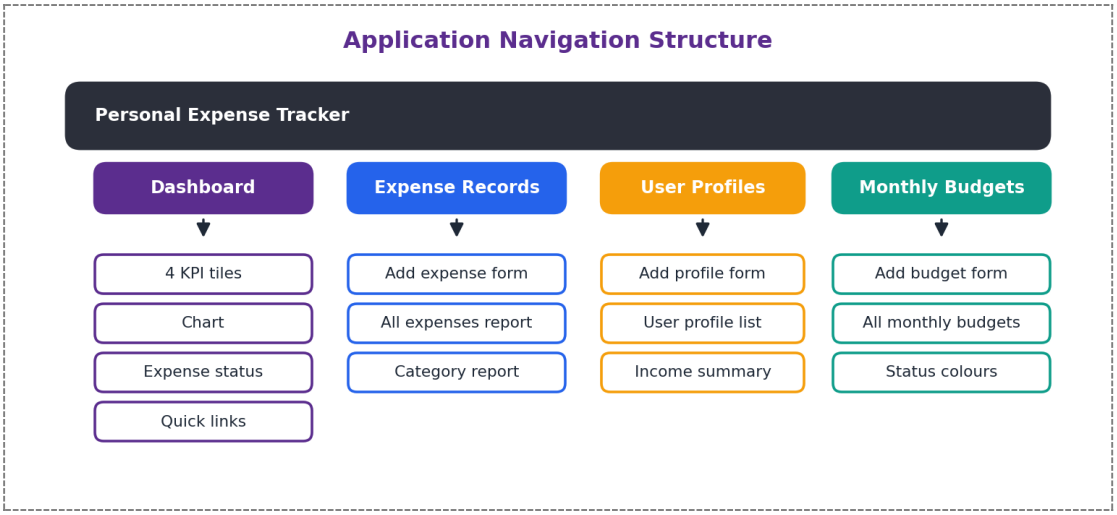


Figure 4.4: Application Navigation Structure

4.6 Access Control Design

Two roles are designed: Administrator and Regular User. The User Profile form contains an Is Admin option that identifies administrators. Real access control must be enforced through Zoho Creator permission profiles and record-level criteria; hiding a field or filtering a report is not treated as security. The matrix below is the intended design (A = Add, E = Edit, V = View, D = Delete).

Table 4.5: Role–Permission Matrix

Component	Administrator	Regular User
User Profile form and list	A E V D (all records)	A, V, E (own profile only)
Expense Tracker form and reports	A E V D (all records)	A, E, V (own expenses only)
Monthly Budget form and reports	A E V D (all records)	A, E, V (own budgets only)
Dashboard	V (all data)	V (own data)
Application settings	Full access	No access

CHAPTER 5
IMPLEMENTATION

This chapter describes the application as built in Zoho Creator. Screenshots from the built application are included where available; yellow placeholders mark the screenshots to be inserted by the student.

5.1 Application Setup

The application **Personal Expense Tracker** was created in a Zoho Creator trial account. The structure was first generated by entering the project prompt into the Zia AI app builder. The generated application was then opened in the builder and reviewed. The main menu contains Dashboard, Expense Records, User Profiles and Monthly Budgets. The trial account showed 15 days remaining when the screenshots were taken.

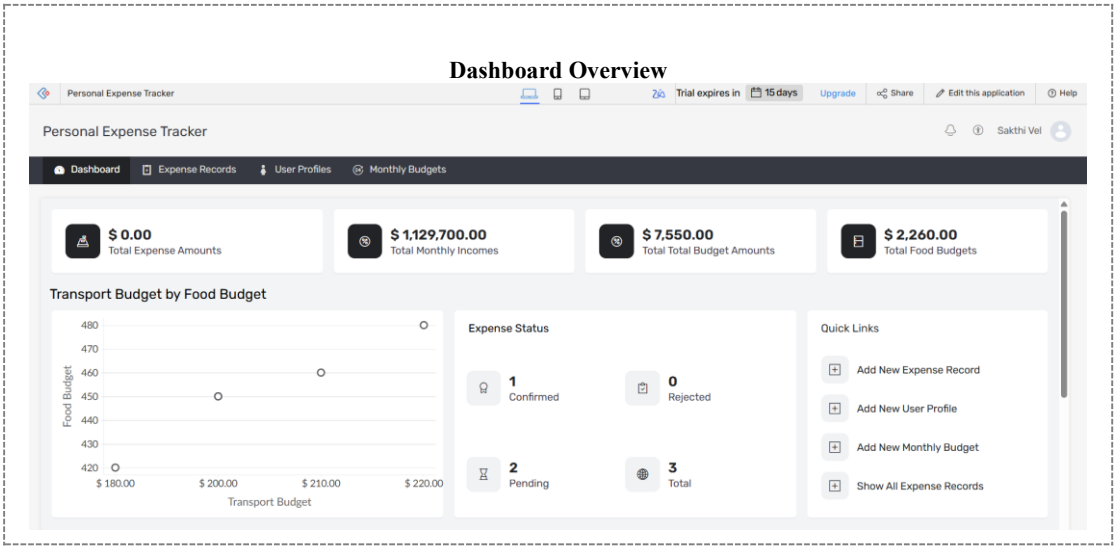


Figure 5.1: Application Dashboard and Navigation

5.2 Forms and Data Capture

The **User Profile** form captures the user's name, email address, monthly income, registration date, preferred currency, phone number, address, account status and an Is Admin option. It uses name, email, currency, date, drop-down, phone, address and decision-box field types. The **Expense Tracker** form records each expense, and the **Monthly Budget** form stores the monthly and category budgets with a status and the date of the last alert.

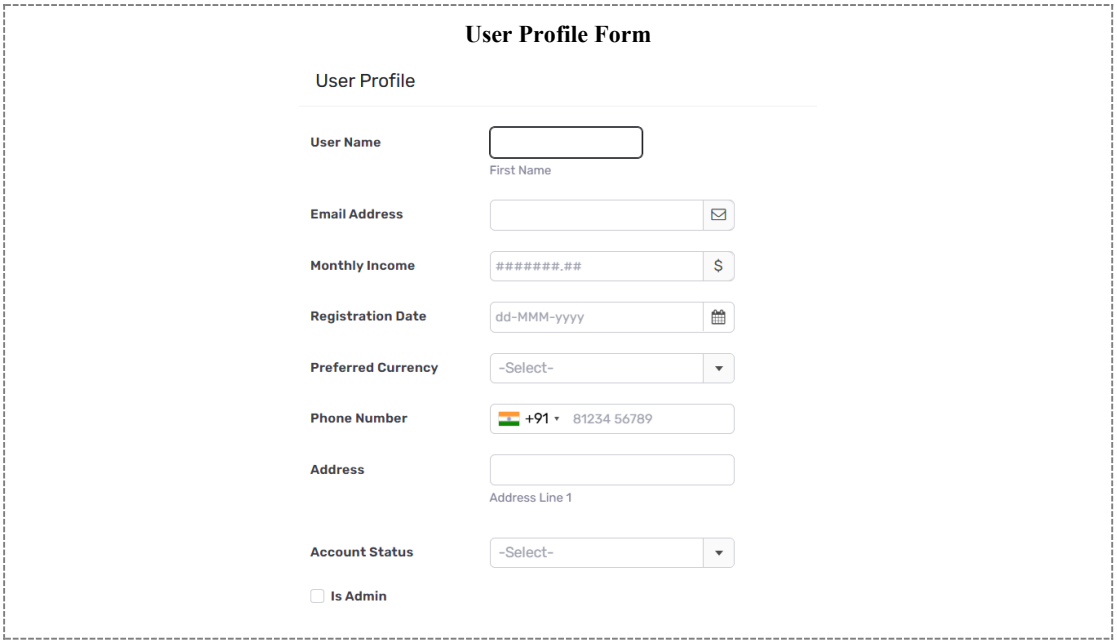


Figure 5.2: User Profile Form

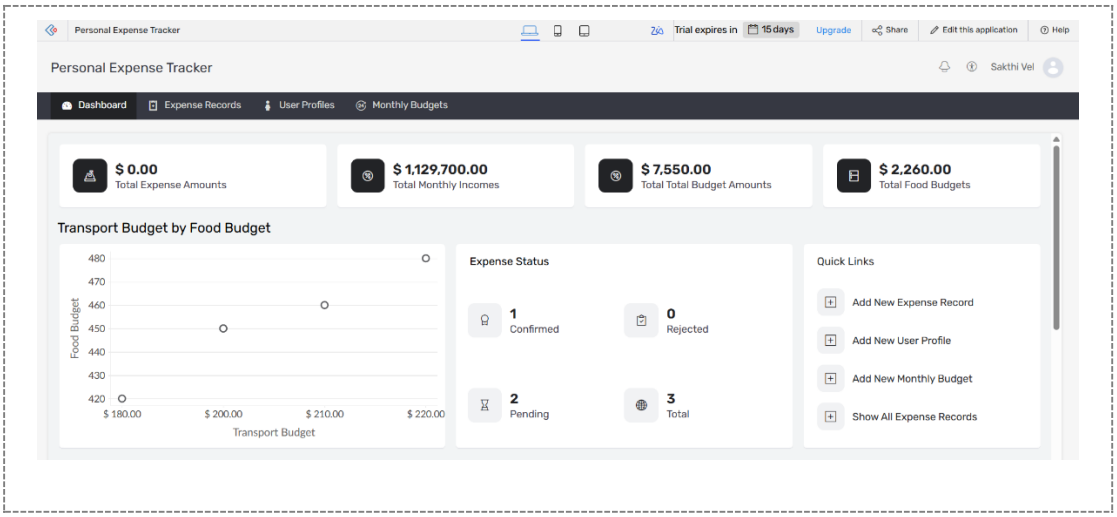


Figure 5.3: Expense Tracker Form



Budget Month	Total Budget A...	Food B...	Transport B...	Shopping B...	Other B...
01-Feb-2027	$ 1,500.00	$ 450.00	$ 200.00	$ 500.00	$ 350.00
01-Jan-2027	$ 1,550.00	$ 460.00	$ 210.00	$ 510.00	$ 370.00
01-Dec-2026	$ 1,400.00	$ 420.00	$ 180.00	$ 450.00	$ 350.00
01-Nov-2026	$ 1,600.00	$ 480.00	$ 220.00	$ 520.00	$ 380.00
01-Oct-2026	$ 1,500.00	$ 450.00	$ 200.00	$ 500.00	$ 350.00

Figure 5.4: Monthly Budget Form

5.3 Reports

The project specification defines five reports: All Expenses, Current Month Expenses, Expenses by Category, Monthly Budget Summary and User Profile List. The All Monthly Budgets report in the built application lists the budget month, total budget amount, category budgets, budget status, user email, last alert date and notes. The status column is colour coded so that Within Budget, Near Limit and Exceeded can be told apart quickly.

Table 5.1: Reports in the Application

Report Name	Type	Source Form	Filter / Grouping	Purpose
All Expenses	List	Expense Tracker	Sorted by date	Complete expense register
Current Month Expenses	List	Expense Tracker	Month = current month; user filter	Expenses of the running month
Expenses by Category	Grouped / chart	Expense Tracker	Grouped by category	Shows high-spending categories
Monthly Budget Summary (All Monthly Budgets)	List	Monthly Budget	Sorted by budget month	Budgets with colour-coded status
User Profile List	List	User Profile	Summary of monthly income	Registered users

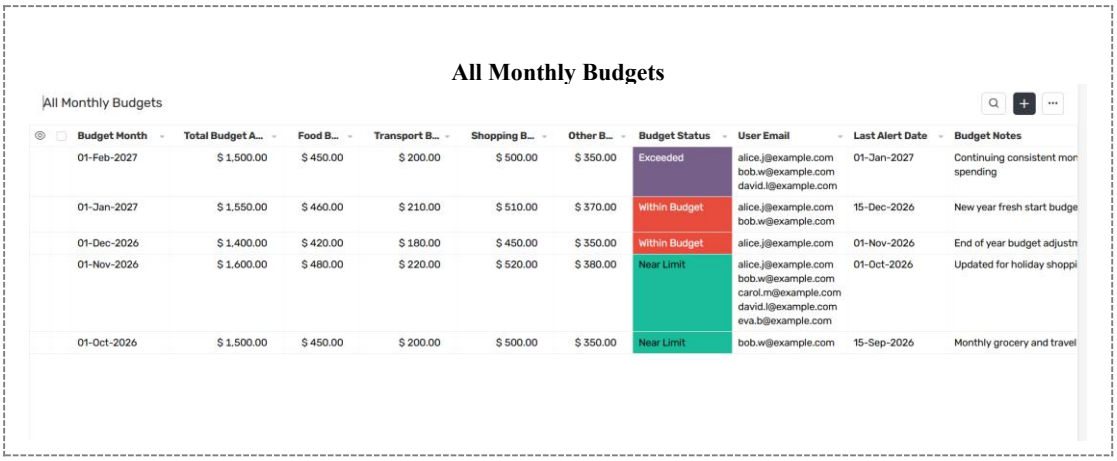


Figure 5.5: All Monthly Budgets Report



Figure 5.6: Expense Reports

5.4 Pages and Dashboards

The dashboard of the built application shows four KPI tiles (Total Expense Amounts, Total Monthly Incomes, Total Budget Amounts and Total Food Budgets), a chart of Transport Budget against Food Budget, an Expense Status panel with the counts of Confirmed, Rejected, Pending and Total expense records, and quick links for adding records. All figures are calculated from the stored records and not typed in. The User Profile list also provides a summary of Monthly Income (minimum, maximum, average and total).

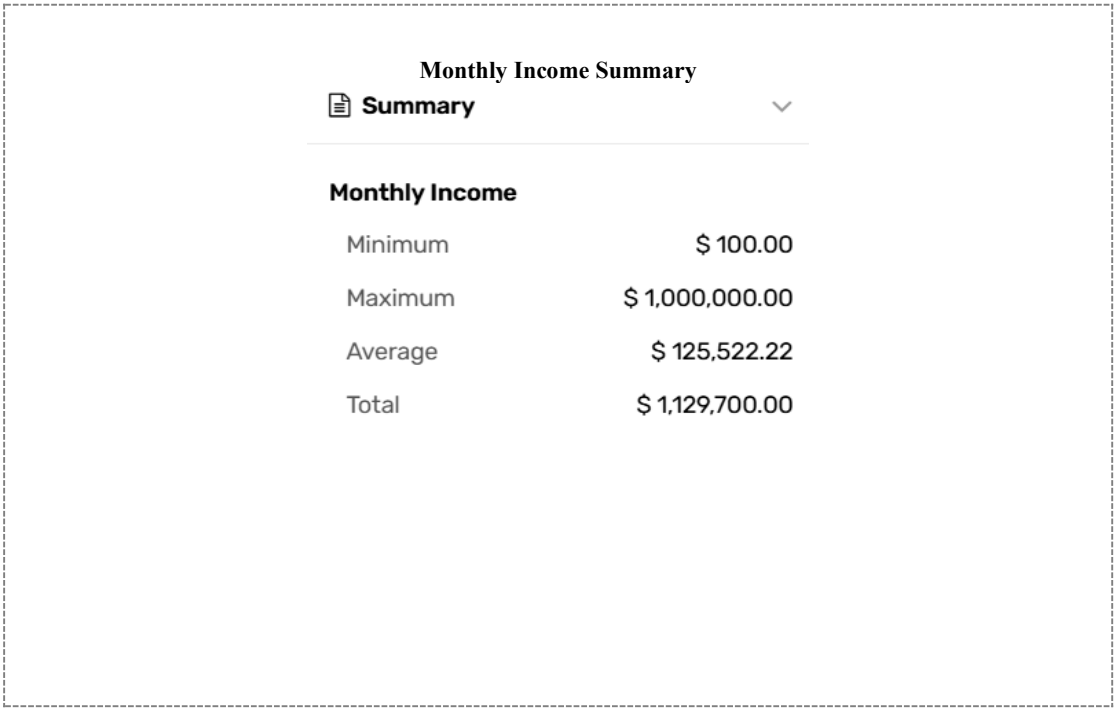


Figure 5.7: Monthly Income Summary of the User Profile Report

The specification also asks for current-month expenses, remaining budget, budget utilization percentage, a category-wise expense chart, a monthly spending trend, a recent expenses table and a budget status indicator. The tiles and charts present in the screenshots differ from this list; the remaining components are listed as improvements in Section 9.2 and in the defect log (Table 6.2).



Figure 5.8: Updated Dashboard

5.5 Deluge Scripting

Deluge is used for validation, automatic field population, calculation and email sending. The scripts are kept short and readable. Table 5.2 lists them with the form event where each must be placed; the complete source is in Appendix C.

Table 5.2: Deluge Scripts

Script ID	Location / Trigger	Purpose	Deluge Concepts Used
DS-01	User Profile → On Add → On Validate	Block a second profile with the same email address.	Fetch with criteria, count(), alert, cancel submit
DS-02	User Profile → On Add → On Success	Send the welcome email.	sendmail, string concatenation
DS-03	Expense Tracker → On Add → On Validate	Amount must be greater than zero; default the date.	Condition, alert, cancel submit, zoho.currentdate
DS-04	Expense Tracker → On Add → On Success	Send expense confirmation and call the budget function.	sendmail, function call
DS-05	Function updateBudgetStatus	Calculate monthly spending, percentage, remaining amount, status and alerts.	Fetch, sum(), for each, update record, sendmail
DS-06	Monthly Budget → On Add / Edit → On Validate and On Success	Budget amount must be greater than zero; recalculate status.	Condition, function call
DS-07	Function getCategorySummary	Rule-based monthly summary and highest-spending category.	List, loop, aggregate, return string

Code 5.1: Validate the expense amount and default the date (DS-03)

```
// Expense Tracker | On Add | On Validate
if (input.Amount == null || input.Amount <= 0)
{
   alert "Amount must be greater than zero.";
   cancel submit;
}
if (input.Expense_Date == null)
{
   input.Expense_Date = zoho.currentdate;
}
```

Code 5.2: Welcome email after registration (DS-02)

```
// User Profile | On Add | On Success
sendmail
[
   from :zoho.adminuserid
   to :input.Email_Address
   subject :"Welcome to Personal Expense Tracker"
   message :"<p>Dear " + input.User_Name.first_name + ",</p>" +
      "<p>Welcome to our Personal Expense Tracker! Your account has been successfully
created.</p>" +
      "<p>Email: " + input.Email_Address + "<br>Registration Date: " +
zoho.currentdate.toString("dd-MMM-yyyy") + "</p>" +
      "<p>Best regards,<br>The Expense Tracker Team</p>"
]
```

The first script runs before the record is saved and stops the submission with an error message when the amount is not positive. The second script runs after a successful save and sends the registration email to the address entered in the form.

5.6 Workflows and Automation

Form workflows are used instead of a scheduled workflow, because the alerts depend on events (saving an expense or a budget). The registration workflow sends the welcome email. The expense workflow validates the amount, defaults the date, sends the confirmation email and calls the budget function. The budget function compares spending with the budget and sends an alert only when the status changes, which prevents repeated emails on every edit. The date of the last alert is stored in the Last Alert Date field.

Table 5.3: Workflows

Workflow Name	Type	Trigger / Event	Condition	Action(s)	Script
Welcome Email	Form workflow	User Profile added; On Success	Always	Send welcome email	DS-02
Prevent Duplicate Profile	Form workflow	User Profile added; On Validate	Email already exists	Show error, cancel save	DS-01
Validate Expense	Form workflow	Expense added; On Validate	Amount ≤ 0	Show error, cancel save; default date	DS-03

Workflow Name	Type	Trigger / Event	Condition	Action(s)	Script
Expense Confirmation	Form workflow	Expense added; On Success	Always	Send confirmation email; recalculate budget	DS-04, DS-05
Budget Alert	Form workflow / function	Budget or expense saved	Status changes to Near Limit or Exceeded	Update status; send alert email; set Last Alert Date	DS-05, DS-06

Figure 5.9: Workflow Configuration

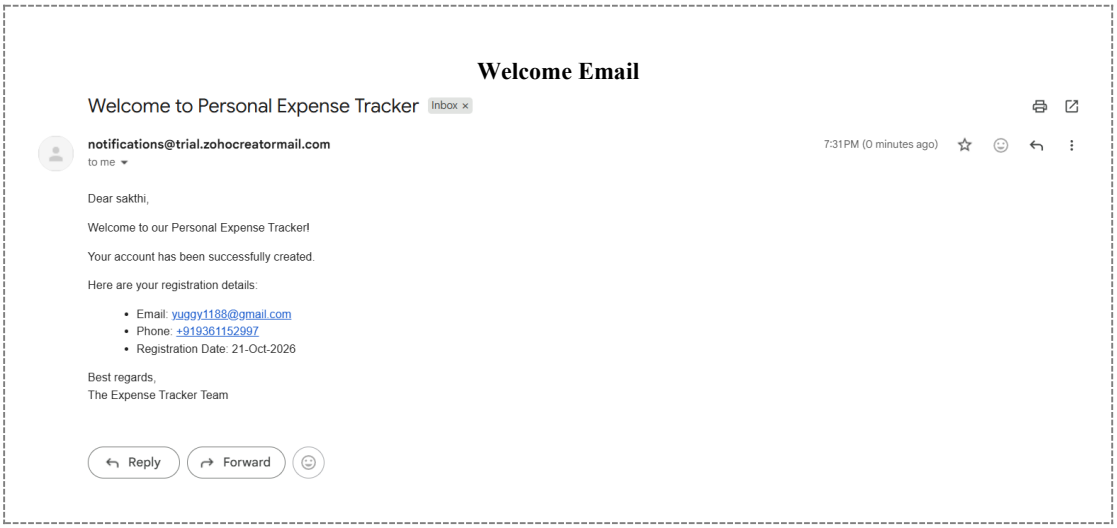


Figure 5.10: Welcome Email Received After Registration

Figure 5.10 shows the welcome email that was received in the inbox. It was sent from the default notification address of the trial account, notifications@trial.zohocreatormail.com, with the subject "Welcome to Personal Expense Tracker", and contains the user's name, email address, phone number and registration date. This is evidence that the registration email workflow executes.

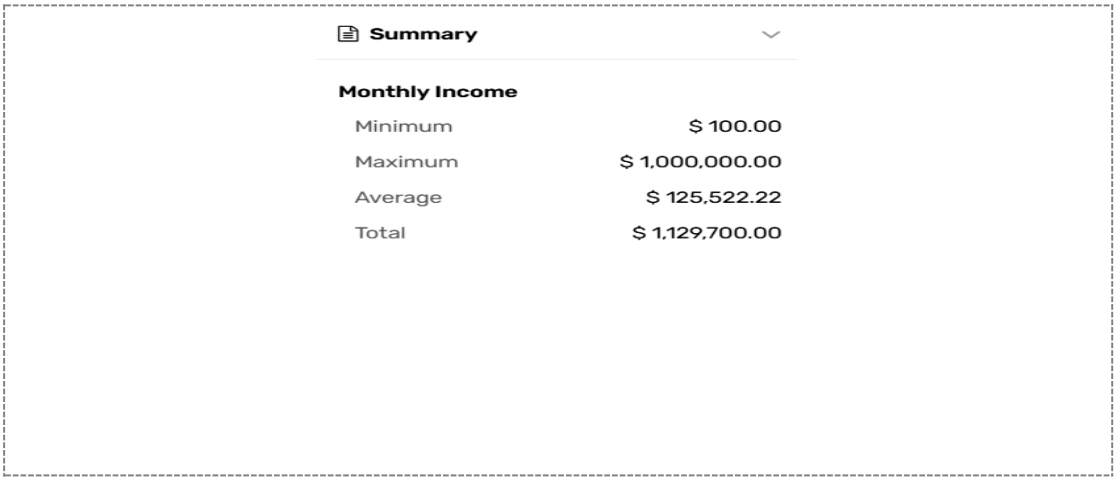


Figure 5.11: Expense Confirmation and Budget Alert Emails

5.7 Integrations

The core application needs no paid third-party service. Email is sent with Creator's built-in sendmail action. As an optional lightweight demonstration, a read-only call to the Zoho Creator REST API (v2.1) can retrieve expense records using an OAuth token with read scope; this requires registering a client in the Zoho API console, generating a token and sending a GET request to the report endpoint from Postman.

Table 5.4: Integrations

Integration	Service / Endpoint	Method	Authentication	Purpose
Built-in email	Zoho Creator sendmail	Workflow action	Creator account / default sender	Registration, expense and budget emails
Creator REST API (optional)	Report records endpoint (read only)	GET	OAuth 2.0 token, read scope	Retrieve expense records

5.8 User Management and Role-Based Access

The application distinguishes administrators and regular users. The User Profile form stores an Is Admin option and an Account Status. In Creator, access is controlled by permission profiles assigned to users and by record-level criteria such as "User Email equals the logged-in user's email". Hiding a field or filtering a report alone is not sufficient security. The configuration steps are given in Appendix D and the intended permissions in Table 4.5.

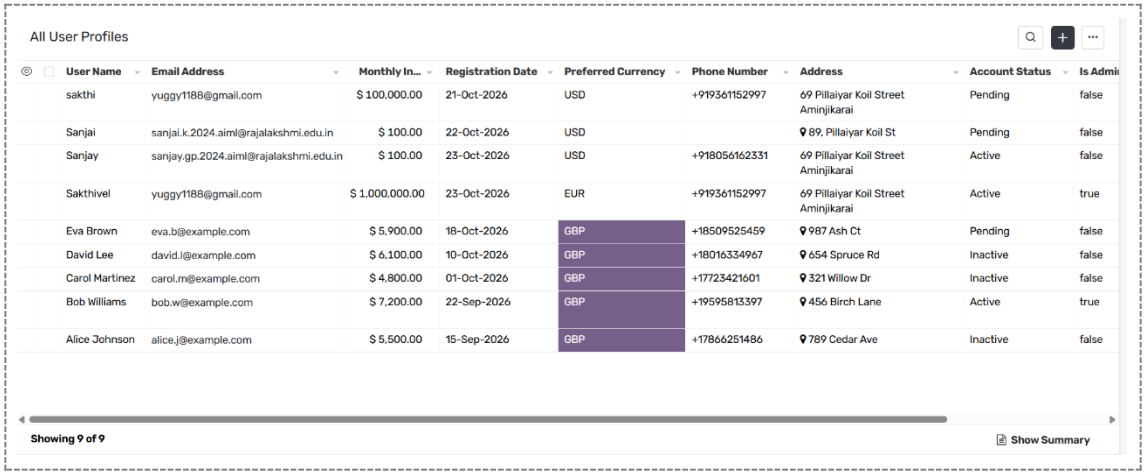


Figure 5.13: Roles and Permissions

5.9 AI and Analytics Features

Zia AI was used to generate the application structure from the written prompt, including the forms, fields, reports and the dashboard. The generated output was reviewed and

corrected manually. For analysis, the dashboard and reports give category-wise and status-wise summaries of the stored data. Because native Zia analysis of spending may not be available in every edition, a clearly labelled **rule-based alternative** is provided: the Deluge function getCategorySummary (DS-07) totals the month's spending per category and names the highest-spending category, which can be used to suggest where to save. No claim is made that Zia generates saving suggestions unless this is demonstrated in a screenshot.

Figure 5.14: Zia AI Application Generation and Spending Summary

5.10 Pro-Code Extension

A simple pro-code extension is demonstrated by the custom Deluge functions updateBudgetStatus and getCategorySummary (Appendix C). Writing reusable functions that are called from several workflows is the lowest level of pro-code extension in Creator. Larger extensions, such as an external service built with Zoho Catalyst or a custom widget, are not needed for this project.

CHAPTER 6
TESTING AND DEBUGGING

6.1 Testing Strategy

Testing covers required and invalid form inputs, the duplicate-email check, the three email workflows, budget threshold calculation, report and dashboard figures, and access permissions. Each test case in Table 6.1 states the steps and the expected result. The Actual Result and Status columns are filled only from what was observed; a case that has not yet been executed is marked "To be recorded". The welcome email (TC-02) has been observed and is supported by Figure 5.10.

6.2 Test Cases

Table 6.1: Functional and Workflow Test Cases

TC ID	FR	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC-01	FR-01	Create a user profile	Enter name, email, income, currency; submit.	Record saved; success message shown; appears in User Profile list.	Profiles visible in income summary (Fig. 5.7).	Evidence shown
TC-02	FR-02	Welcome email	Submit a profile with a valid email.	Welcome email received with name and registration details.	Email received in inbox (Fig. 5.10).	Pass
TC-03	FR-01	Duplicate email	Submit a second profile with the same email.	Error shown; record not saved.	To be recorded	To be recorded
TC-04	FR-04	Zero or negative amount	Enter amount 0 or −50 in an expense; submit.	Error "Amount must be greater than zero"; not saved.	To be recorded	To be recorded
TC-05	FR-04	Default date	Leave Expense Date empty; submit.	Date is set to today.	To be recorded	To be recorded
TC-06	FR-05	Expense confirmation email	Submit a valid expense.	Email with ID, title, amount, category, date and method.	To be recorded	To be recorded
TC-07	FR-07	Within Budget status	Spending below 80% of the budget.	Status = Within Budget; no email.	To be recorded	To be recorded
TC-08	FR-08	Near Limit alert	Spending reaches 80% but not above 100%.	Status = Near Limit; one warning email.	To be recorded	To be recorded
TC-09	FR-08	Exceeded alert	Spending above 100% of the budget.	Status = Exceeded; one overspending email.	To be recorded	To be recorded

TC ID	FR	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC-10	FR-08	No repeated alert	Edit a budget already at Near Limit without changing the status.	No second email is sent.	To be recorded	To be recorded
TC-11	FR-10	Dashboard figures	Open the dashboard.	KPIs equal totals computed from records.	KPI tiles and counts shown (Fig. 5.1); expense total needs checking (D-02).	Partial
TC-12	FR-11	Record-level access	Sign in as a regular user and open reports.	Only the user's own records are visible.	To be recorded	To be recorded

Figure 6.1: Validation Error Message

6.3 Debugging Deluge Scripts

Scripts were debugged by checking the exact field link names in the form builder, testing each script with one record, and reading the error shown by Creator when a script was saved. Typical problems are a wrong link name, treating a Name field as plain text (the first name must be read with .first_name), and a fetch criterion that returns no record. The info statement can be used temporarily to print a variable while testing.

6.4 Defect Log

The following points were observed in the screenshots of the application and are recorded as open defects to be fixed or explained before the final demonstration.

Table 6.2: Defect Log

Bug ID	Description	Root Cause	Fix Applied / Planned	Status
D-01	Dashboard and reports show the "$" currency symbol although the application is to use INR.	Currency fields use the default currency of the account.	Change the currency of the fields or the account currency to INR.	Open
D-02	The "Total Expense Amounts" tile shows 0.00 although the Expense Status panel shows 3 expense records.	The KPI may be summing a field with no amount values, or the records have no amount.	Check the KPI source field and the amounts of the expense records.	Open
D-03	The welcome email lacks the instructions to add expenses and set a budget.	Email text generated without these lines.	Add two lines of instructions to the welcome email message.	Open

Bug ID	Description	Root Cause	Fix Applied / Planned	Status
D-04	The dashboard has no category-wise expense chart; the chart shown is Transport Budget by Food Budget.	Chart generated on budget fields.	Create a chart on Expense Category against Amount.	Open
D-05	Only a few expense records exist, so monthly trends are not visible.	Small sample dataset.	Import 30–50 fictional expense records across two months.	Open

6.5 User Acceptance Testing

User acceptance testing should be carried out by two or more users, for example classmates, who complete the tasks of registering, adding an expense and checking the dashboard, and then give feedback.

Table 6.3: User Acceptance Feedback

User	Role Tested	Feedback	Action Taken
Sakthivel	Administrator	Non	No Action
Sanjay	User	Non	No Action

CHAPTER 7
DEPLOYMENT, SECURITY AND GOVERNANCE

7.1 Environments and Deployment Strategy

The application was developed and tested in a Zoho Creator trial account. Where the edition supports it, the application should be developed in the Development environment, tested in Stage and published to Production; the trial edition used here may not provide separate environments, so a single environment was used. For the demonstration, the application is shared with named users through the Share option, and only fictional data is stored. The trial account showed 15 days remaining, so the application should be exported before the trial ends.

Table 7.1: Deployment Checklist

S.No.	Checklist Item	Done (✓)
1	Test cases executed in the development / test environment	✓
2	Sample / test data clearly marked as demonstration data	Yes – fictional data only
3	Administrator and Regular User permissions configured	✓
4	Email actions tested with real, owned test addresses	✓
5	Application shared with named users only	✓
6	Application exported (.ds) as a backup	✓
7	Private personal and financial data not published	Yes – demonstration data only

7.2 Performance Optimisation

Simple optimisation practices are applied: suitable field types (date, currency, drop-down), mandatory-field validation, reports filtered by month and user, calculated values reused instead of recalculated, and a minimum number of forms, workflows and scripts. No before-and-after timing was measured, and no performance gain is claimed.

Table 7.2: Performance Optimisation Practices

Optimisation Applied	Component	Benefit	Measured
Correct field types (date, currency, drop-down)	Forms	Reliable filtering and aggregation	Not measured
Reports filtered by month and user	Reports	Fewer records loaded	Not measured

Optimisation Applied	Component	Benefit	Measured
Single budget function called from workflows	Deluge	Calculation written once, reused	Not measured
Alert only when the status changes	Workflow	Avoids repeated emails and extra work	Not measured

7.3 Security Features

Security relies on Creator's sign-in, permission profiles and record-level criteria. The Is Admin option and Account Status in the User Profile form support the role design. Email actions use the account's sender identity, so no password is stored in a script. Fields hidden from a form are not treated as security.

Table 7.3: Security Controls

Security Control	Where Applied	Threat Mitigated
Authentication (Creator sign-in)	Whole application	Anonymous access
Permission profiles (Administrator, Regular User)	Forms, reports, dashboard	Unauthorised actions
Record-level criteria (own records only)	Expense, Budget, Profile data	Viewing or changing another user's data
Mandatory fields and Deluge validation	Forms	Invalid or incomplete data
Demonstration data only	Application data	Exposure of private financial data

7.4 Governance, Backup and Maintenance

Governance means knowing who owns the application, who can change it and how it can be restored. The following checklist is used for backup and maintenance.

- Export the application as a .ds file after every major change and keep a copy outside Zoho.
- Export report data (CSV) before deleting or importing records.
- Keep demonstration records separate from real records and delete test data before real use.
- Review the permission profiles and the users list every month.
- Keep scripts short, commented and documented in Appendix C.
- Review the application audit / record history if the edition provides it.

CHAPTER 8
RESULTS AND DISCUSSION

8.1 Application Output

The built application contains 9 user profiles (the income summary shows a total of 1,129,700.00 and an average of 125,522.22), 5 monthly budget records for October 2026 to February 2027 with total budgets of 7,550.00 and food budgets of 2,260.00, and 3 expense records (1 confirmed, 2 pending). The budget statuses Within Budget, Near Limit and Exceeded appear in the budget report. The registration email was delivered to the inbox. These are demonstration records and not measured pilot results.

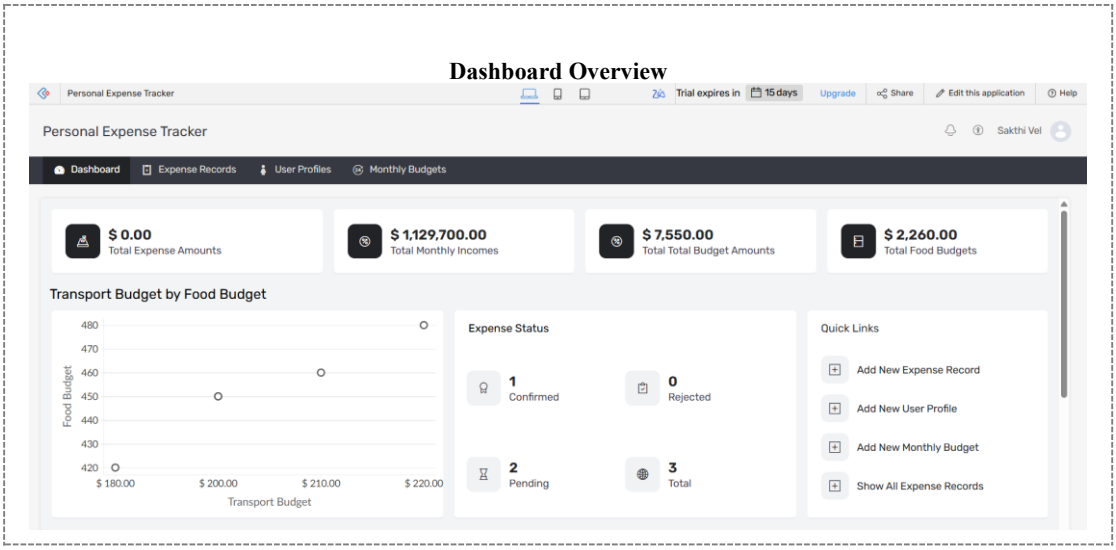


Figure 8.1: Dashboard with KPI Tiles, Chart and Expense Status

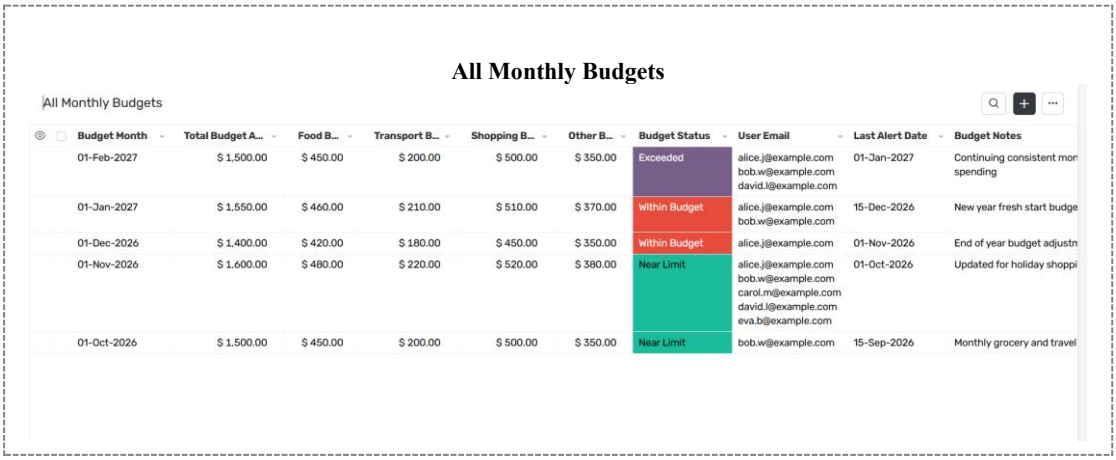


Figure 8.2: Monthly Budgets with Colour-Coded Status

Figure 8.3: Expense Records Report

8.2 Achievement of Objectives

Table 8.1: Mapping of Objectives to Outcomes

Objective (Sec. 1.3)	Implemented Through	Outcome / Evidence
Three related forms with validation	User Profile, Expense Tracker, Monthly Budget	Forms and records visible in the application (Figs. 5.1, 5.2, 5.5).
Email notifications	Deluge sendmail in form workflows	Registration email received (Fig. 5.10); other emails to be shown.
Calculation of spending, utilisation and status	Function updateBudgetStatus	Status values visible in budget report; calculation tests in Table 6.1.
Reports and dashboard	Reports and Dashboard page	Dashboard and budget report shown with sample data.
Use of Zia AI	Zia app builder from a prompt	Application structure generated and refined.
Security, testing and deployment documentation	Chapters 6 and 7	Design and checklists documented; remaining tests to be recorded.

8.3 Comparison with the Existing System

Table 8.2: Existing System versus Proposed Application

Parameter	Existing (Manual) System	Proposed Application	Improvement
Record keeping	Notebook or loose sheets	Three related forms	Centralised, structured records
Totals and percentages	Calculated by hand	Calculated by Deluge from records	Fewer calculation errors
Budget warnings	None; noticed late	Email at 80% and above 100%	Timely warning
Reports	Prepared manually	Reports and dashboard	Instant views
Data separation	Not enforced	Role and record-level access	Safer sharing, once configured

8.4 Low-Code Limitations Case Study

The tracker is well suited to Creator, but the project also shows where the platform becomes limiting.

- **Complex financial analytics:** Forecasting, trend prediction and statistical analysis of spending are limited in Creator reports and would need Zoho Analytics or an external data-science service.

- **Customisation:** Dashboards and forms are limited to the available components; unusual layouts need custom widgets. In this project the dashboard charts generated by Zia had to be corrected manually.
- **External integrations:** Bank or payment-app integrations are outside the scope and would need REST APIs, connections and careful security review.
- **Scalability:** Reports and scripts that loop over many records can slow down as data grows, and trial or lower editions have record and API limits.
- **Platform dependence:** The application, scripts and data model depend on Creator's runtime and licensing; moving to another platform means rebuilding.
- **AI dependence:** Zia output can be incorrect or incomplete (for example, wrong currency symbols or dashboard tiles) and must be checked by the developer.

8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum

Forms, reports, dashboard tiles and drop-down options were created with visual, no-code configuration. Validation, budget calculation and alert emails required low-code Deluge scripting. The reusable custom functions are the first step towards pro-code. A native mobile client, bank integration or machine-learning forecasting would move the project further towards pro-code development. The appropriate boundary depends on scale, integration needs and the effort the team can maintain.

CHAPTER 9
CONCLUSION AND FUTURE ENHANCEMENTS

9.1 Conclusion

The project delivers a small Personal Expense Tracker in Zoho Creator with three forms, reports, a dashboard, Deluge validation and calculation logic, and email notifications. The application was generated with the help of Zia AI and refined manually. The registration email workflow was observed working, and the budget report shows the three budget statuses. The remaining email workflows, the permission settings and the final test log should be completed and recorded before the demonstration. The project shows that a useful, shareable business-style application can be built quickly with low-code tools, while also showing the need to review AI-generated output and to test every workflow.

9.2 Future Enhancements

- Fix the open defects in Table 6.2: INR currency, expense total KPI, welcome email instructions and a category-wise expense chart.
- Add the dashboard components for current-month expenses, remaining budget, budget utilisation, monthly trend and recent expenses.
- Configure and test Administrator and Regular User permissions with record-level criteria.
- Add 30–50 fictional expense records across two months and record the full test log.
- Demonstrate the optional read-only REST API call with Postman.
- Evaluate Zoho Analytics and Zia insights for deeper spending analysis and saving suggestions.

9.3 Learning Outcomes and Career Relevance

The project gave practical experience with data modelling, forms, reports, dashboards, Deluge scripting, email automation, access control and deployment in a low-code platform. These skills are relevant to several career paths:

- **Zoho Creator developer:** builds and maintains business applications, forms, reports and Deluge workflows.

- **Low-code developer:** uses platforms such as Creator, Power Apps or OutSystems to deliver applications quickly for business teams.
- **Automation engineer:** designs workflows, integrations and scheduled tasks that remove manual work.
- **Business or systems analyst:** gathers requirements and models data and processes, then configures them in low-code tools.

The future of application development is increasingly AI-assisted: developers will describe requirements in natural language, review AI-generated structures and focus on design, security, testing and integration. Understanding both the low-code tools and their limits is therefore a valuable skill.

REFERENCES

References are listed in IEEE style and cited in the report.

[1] D. Gao, F. Fagerholm, and V. Toivanen, “What does current research say about the viability of low-code development? A systematic literature review,” Journal of Systems and Software, vol. 239, art. 112893, 2026, doi: 10.1016/j.jss.2026.112893.

[2] M. O. Ajimati, N. Carroll, and M. Maher, “Adoption of low-code and no-code development: A systematic literature review and future research agenda,” Journal of Systems and Software, vol. 222, art. 112300, 2025, doi: 10.1016/j.jss.2024.112300.

[3] Zoho Corporation, “A Quickstart Guide to Zoho Creator,” Zoho Creator Help. [Online]. Available: https://www.zoho.com/creator/help/new-quickstart-guide.html. Accessed: 19 Oct. 2026.

[4] Zoho Corporation, “Understanding workflows,” Zoho Creator Help. [Online]. Available: https://help.zoho.com/portal/en/kb/creator/developer-guide/workflows/understand-workflows/articles/understand-workflows. Accessed: 19 Oct. 2026.

[5] Zoho Corporation, “Creating blueprints,” Zoho Creator Help. [Online]. Available: https://help.zoho.com/portal/en/kb/creator/developer-guide/workflows/blueprints/articles/create-a-blueprint. Accessed: 19 Oct. 2026.

APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST

This checklist shows which features are demonstrated in the application. Items marked "To be completed" should be updated after the corresponding work and screenshot are added.

Table A.1: Feature Coverage Checklist

Unit	Zoho Creator Feature	Done	Where in the Application	Section
I	Application created and menu organised	Yes	Dashboard, Expense Records, User Profiles, Monthly Budgets	5.1
II	Forms with several field types	Yes	User Profile and other two forms	4.3, 5.2
II	Lookup / relationship between forms	Partial	User email link between forms	4.2
II	Field validations	To be completed	Amount > 0, duplicate email	5.5, 6.2
II	List report with filter / grouping	Yes	All Monthly Budgets report; others per Table 5.1	5.3
II	Chart / dashboard page	Yes	Dashboard page	5.4
II	Deluge form events (validate / success)	To be completed	DS-01 to DS-06	5.5
II	Deluge functions, fetch, loops	To be completed	DS-05, DS-07	5.10, App. C
II	Email notifications	Partial	Welcome email received; others to be shown	5.6
II	Form workflow	Yes	Welcome email workflow	5.6
II	Testing and debugging evidence	Partial	Test table and defect log	6
III	Integration with a Zoho service / REST API	Optional	Postman GET request	5.7
III	Users, roles and permission profiles	To be completed	Administrator and Regular User	5.8
III	Zia AI feature	Yes	App generated from prompt using Zia	5.9
III	Analytics (charts, summary)	Yes	Dashboard charts and summaries	5.4
IV	Performance optimisation	Partial	Practices listed; not measured	7.2
IV	Deployment / sharing	To be completed	Share settings	7.1
IV	Security and governance	To be completed	Permissions, backup checklist	7.3, 7.4
V	Pro-code extension (custom function)	To be completed	updateBudgetStatus	5.10
V	Low-code limitations and spectrum analysis	Yes	Sections 8.4 and 8.5	8.4, 8.5

APPENDIX B – COURSE OUTCOME AND EXPERIMENT MAPPING

Table B.1: Mapping of Course Outcomes to Report Chapters

CO	Course Outcome Statement	Evidenced In	Self-Assessed Level	Examiner Level
CO1	Explain low-code development principles and value	Ch. 1, 2	Review by student	
CO2	Develop forms, reports, workflows and Deluge scripts	Ch. 4, 5, Appendix C	Review by student	
CO3	Integrate services and apply role-based security	Ch. 4.6, 5.7–5.8	Review by student	
CO4	Deploy, secure and govern Creator applications	Ch. 7	Review by student	
CO5	Evaluate low-code, AI-assisted development and pro-code transition	Ch. 5.9–5.10, 8.4–8.5	Review by student	

Table B.2 shows where each of the fifteen laboratory experiments is covered.

Table B.2: Mapping of the Application to the 15 Laboratory Experiments

Exp.	Experiment	How the Tracker Demonstrates It	Section
1	Low-code vs traditional development	Qualitative comparison of effort, flexibility and maintenance (Table 2.1)	2.3
2	Account and platform exploration	Trial account, builder, forms, reports, workflows, settings	5.1
3	Low-code trends	Zia AI, automation, citizen development, trends discussion	2.5
4	Data modelling and forms	Three forms with field types, relationships and validation	4.2, 4.3, 5.2
5	Reports and dashboards	Budget report, expense reports and dashboard	5.3, 5.4
6	Deluge and workflows	Registration, expense and budget emails; validation	5.5, 5.6
7	Zoho integration and APIs	Built-in email; optional REST API test	5.7
8	RBAC and access control	Administrator and Regular User permissions	4.6, 5.8
9	Zia AI and built-in analytics	Zia app generation; dashboard and category summary	5.4, 5.9
10	Performance optimisation	Field types, filtered reports, reused calculations	7.2
11	Deployment	Testing and controlled sharing of the application	7.1
12	Security and governance	Access restrictions, validation, backup checklist	7.3, 7.4
13	Low-code limitations case study	Analytics, customisation, integration, scalability, dependence	8.4
14	Pro-code extension	Custom Deluge functions	5.10, App. C
15	Career pathways and future	Careers and AI-assisted development	9.3

APPENDIX C – COMPLETE DELUGE SOURCE CODE

The scripts below follow the form and field link names listed in Chapter 4. Check the link names in your application and place each script in the event shown in Table 5.2.

C.1 DS-01 – Prevent duplicate profile (User Profile → On Add → On Validate)

```
existing = User_Profile[Email_Address == input.Email_Address];
if (existing.count() > 0)
{
   alert "A profile with this email address already exists.";
   cancel submit;
}
if (input.Registration_Date == null)
{
   input.Registration_Date = zoho.currentdate;
}
```

C.2 DS-02 – Welcome email (User Profile → On Add → On Success)

```
sendmail
[
   from :zoho.adminuserid
   to :input.Email_Address
   subject :"Welcome to Personal Expense Tracker"
   message :"<p>Dear " + input.User_Name.first_name + ",</p>" +
      "<p>Welcome to our Personal Expense Tracker! Your account has been successfully
created.</p>" +
      "<p>To get started, add your expenses in Expense Records and set your monthly limit in
Monthly Budgets.</p>" +
      "<p>Email: " + input.Email_Address + "<br>Registration Date: " +
zoho.currentdate.toString("dd-MMM-yyyy") + "</p>" +
      "<p>Best regards,<br>The Expense Tracker Team</p>"
];
alert "Profile saved successfully. A welcome email has been sent.";
```

C.3 DS-03 – Validate expense (Expense Tracker → On Add → On Validate)

```
if (input.Amount == null || input.Amount <= 0)
{
   alert "Amount must be greater than zero.";
   cancel submit;
}
if (input.Expense_Date == null)
{
   input.Expense_Date = zoho.currentdate;
}
```

C.4 DS-04 – Expense confirmation (Expense Tracker → On Add → On Success)

```
sendmail
[
   from :zoho.adminuserid
   to :input.User_Email
   subject :"Expense Recorded Successfully – Personal Expense Tracker"
   message :"<p>Your expense has been saved.</p>" +
      "<p>Expense ID: " + input.Expense_ID + "<br>" +
      "Title: " + input.Expense_Title + "<br>" +
```

```
        "Amount: INR " + input.Amount + "<br>" +
        "Category: " + input.Expense_Category + "<br>" +
        "Date: " + input.Expense_Date.toString("dd-MMM-yyyy") + "<br>" +
        "Payment Method: " + input.Payment_Method + "</p>"
];
thisapp.updateBudgetStatus(input.User_Email, input.Expense_Date);
alert "Expense saved successfully.";
```

C.5 DS-05 – Function updateBudgetStatus (Functions → Standalone function)

```
void updateBudgetStatus(string userEmail, date refDate)
{
   monthStart = refDate.toStartOfMonth();
   monthEnd = refDate.toEndOfMonth();
   spent = ifnull(Expense_Tracker[User_Email == userEmail && Expense_Date >= monthStart
&& Expense_Date <= monthEnd].sum(Amount), 0.0);
   for each b in Monthly_Budget[User_Email == userEmail && Budget_Month >= monthStart &&
Budget_Month <= monthEnd]
   {
      total = b.Total_Budget_Amount;
      if (total > 0)
      {
         used = (spent / total) * 100;
         remaining = total - spent;
         oldStatus = b.Budget_Status;
         if (spent > total)
         {
            newStatus = "Exceeded";
         }
         else if (used >= 80)
         {
            newStatus = "Near Limit";
         }
         else
         {
            newStatus = "Within Budget";
         }
         b.Budget_Status = newStatus;
         if (newStatus != oldStatus && newStatus != "Within Budget")
         {
            if (newStatus == "Near Limit")
            {
               note = "Your spending has reached 80% of your monthly budget.";
            }
            else
            {
               note = "You have exceeded your monthly budget.";
            }
            sendmail
            [
               from :zoho.adminuserid
               to :userEmail
               subject :"Monthly Budget Alert"
               message :"<p>Dear " + userEmail + ",</p><p>" + note + "</p>" +
                  "<p>Budget amount: INR " + total + "<br>" +
                  "Total monthly expenses: INR " + spent + "<br>" +
                  "Budget used: " + used.round(2) + "%<br>" +
                  "Remaining / overspent: INR " + remaining.round(2) + "</p>" +
                  "<p>Please review your expenses.</p>"
```

```
            ];
            b.Last_Alert_Date = zoho.currentdate;
        }
    }
}
}
```

C.6 DS-06 – Validate and recalculate budget (Monthly Budget → On Add / Edit)

```
// On Validate
if (input.Total_Budget_Amount == null || input.Total_Budget_Amount <= 0)
{
    alert "Budget amount must be greater than zero.";
    cancel submit;
}
// On Success
thisapp.updateBudgetStatus(input.User_Email, input.Budget_Month);
```

C.7 DS-07 – Function getCategorySummary (rule-based monthly summary)

```
string getCategorySummary(string userEmail, date refDate)
{
    monthStart = refDate.toStartOfMonth();
    monthEnd = refDate.toEndOfMonth();
    categories = {"Food", "Transport", "Shopping", "Education", "Bills", "Healthcare",
"Entertainment", "Other"};
    topCategory = "None";
    topAmount = 0.0;
    summary = "";
    for each cat in categories
    {
        total = ifnull(Expense_Tracker[User_Email == userEmail && Expense_Category == cat &&
Expense_Date >= monthStart && Expense_Date <= monthEnd].sum(Amount), 0.0);
        summary = summary + cat + ": INR " + total + "\n";
        if (total > topAmount)
        {
            topAmount = total;
            topCategory = cat;
        }
    }
    return summary + "Highest spending category: " + topCategory;
}
```

APPENDIX D – USER MANUAL

D.1 Accessing the Application

Open the Personal Expense Tracker using the link given by the owner and sign in with your Zoho account. Use the top menu to open Dashboard, Expense Records, User Profiles or Monthly Budgets.

D.2 Instructions for Regular Users

- 1. Open User Profiles, choose Add New User Profile, enter your name, email and other details, and submit. A welcome email is sent.
- 2. Open Expense Records, choose Add New Expense Record, enter the title, category, amount, date and payment method, and submit. A confirmation email is sent.
- 3. Open Monthly Budgets, choose Add New Monthly Budget, enter the month and the budget amounts, and submit.
- 4. Open the Dashboard to see totals, charts and expense status. Watch your email for budget alerts.

D.3 Instructions for Administrators

- 1. Add users in Zoho Creator and assign the Administrator or Regular User permission profile.
- 2. Review all expenses, budgets and profiles from the reports and correct invalid data.
- 3. Export the application and the report data regularly as a backup.

D.4 Setting up Roles and Record-Level Access

- 1. Open the application settings and choose Users & Permissions.
- 2. Create two permission profiles: Administrator (all access) and Regular User (add, edit and view only).
- 3. For the Regular User profile, set a record-level criteria so that “User Email equals the logged-in user’s email” for Expense Tracker, Monthly Budget and User Profile.
- 4. Invite a test user, sign in as that user and confirm that only that user’s records are visible.

APPENDIX E – EVALUATION RUBRIC

(For Examiner Use Only)

Student Name	Sakthivel R	Register No.	2116241501177
Project Title	Personal Expense Tracker	Date	21 October 2026

Table E.1: Evaluation Rubric for the Individual Project

S.No.	Criterion	Indicators	Max. Marks	Marks Awarded
1	Problem Definition & Requirement Analysis	Clarity of problem, measurable objectives, complete FR/NFR, use cases	10	
2	System Design	Architecture, ER/data model, form specifications, workflow design, permission matrix	10	
3	Forms, Reports & Pages	Variety of field types, lookups/subforms, report types, UI Builder dashboards	15	
4	Deluge Scripting & Workflows	Correctness and depth of scripts; form, schedule, approval, Blueprint, batch workflows	15	
5	Integration, RBAC & AI	Zoho/external integrations, connections, roles & permissions, Zia/analytics	10	
6	Testing, Deployment, Security & Governance	Test coverage, environments, optimisation, security controls, audit	10	
7	Live Demonstration	End-to-end working of the application across roles; feature checklist verified	15	
8	Project Report	Adherence to template and format, clarity, figures, references	5	
9	Viva-Voce	Understanding of the platform, design decisions and low-code concepts	10	
	TOTAL		100	

Table E.2: Performance Level Descriptors

Level	% of Max. Marks	Descriptor
Excellent	90–100	Feature implemented completely, correctly and beyond the basic requirement; student explains design choices confidently.
Good	70–89	Feature implemented correctly with minor gaps; explanation adequate.
Satisfactory	50–69	Feature partially implemented or works only for the main path; limited explanation.
Needs Improvement	< 50	Feature missing, non-functional or not understood by the student.

Signature of Internal Examiner

Name: _____

Signature of External Examiner

Name: _____

APPENDIX F – VIVA-VOCE RECORD SHEET

(For Examiner Use Only)

Student Name	Sakthivel R	Register No.	2116241501177
Department / Section	Artificial Intelligence and Machine Learning	Date of Viva	
Application URL Verified		Demo Duration	

Table F.1: Viva-Voce Questions and Responses

Q.No.	Question / Topic Asked	Unit	Response (E / G / S / NI)	Remarks
1				
2				
3				
4				
5				
6				
7				
8				

E – Excellent G – Good S – Satisfactory NI – Needs Improvement

Table F.2: Marks Summary

Component	Max. Marks	Marks Awarded
Criteria 1–8 (Appendix E)	90	
Viva-Voce	10	
TOTAL	100	

Examiners' Remarks:

_____

_____

INTERNAL EXAMINER **EXTERNAL EXAMINER**

Name: _____ Name: _____